

SDE PORTFOLIO PROJECT

DeployX Interview Guide

A mini cloud deployment platform that turns a Git repository into a monitored, containerized application.

Java + Spring Boot · React + TypeScript · PostgreSQL · RabbitMQ · Redis · Docker · Nginx · Prometheus · Grafana

30-Second Pitch

DeployX is a mini cloud deployment platform inspired by services such as Render and Railway. A developer connects a public Git repository, creates a project, and starts a deployment. DeployX asynchronously clones the repository, builds a Docker image, runs the application with resource limits, performs a health check, stores logs, and exposes the application through a generated URL.

It demonstrates authentication, REST APIs, PostgreSQL persistence, asynchronous jobs, RabbitMQ, Redis coordination, Docker isolation, reverse proxying, observability, CI/CD, and cloud deployment architecture.

The Problem

Manual deployment requires developers to configure servers, install dependencies, build images, start containers, inspect logs, monitor health, and recover from broken releases. DeployX reduces that workflow to:

```
Connect repository -> Deploy -> Monitor -> Access URL -> Roll back
```

Architecture

```

Developer
|
React Dashboard
|
Nginx Reverse Proxy
|
Spring Boot API ----- PostgreSQL
|           \----- Redis
| \
|  RabbitMQ Queue
|       |
|  Deployment Worker ---- Git Repository
|                           Docker Runtime
|                           Health Check
|
Prometheus ----- Grafana
  
```

End-to-End Workflow

1. User registers or logs in and receives a JWT.
2. User creates a project with repository URL and branch.
3. API verifies ownership and inserts a deployment with status QUEUED.
4. API publishes the deployment ID to RabbitMQ and returns immediately.
5. Worker consumes the message and Redis prevents duplicate processing.
6. Worker clones the branch, validates the Dockerfile, builds an image, and starts a container.
7. Worker applies CPU and memory limits and probes the configured HTTP health path.
8. Deployment becomes READY or FAILED; logs are persisted in PostgreSQL.
9. Dashboard polls deployment status and displays logs and the public URL.

Technology Decisions

Spring Boot

REST APIs, dependency injection, transactions, security, persistence, and concurrency.

PostgreSQL

Durable source of truth for users, projects, deployments, logs, and audit records.

RabbitMQ

Decouples slow Docker work from the HTTP request and allows worker scaling.

Redis

Stores short-lived coordination state and per-deployment processing locks.

Docker

Isolates application workloads with unique containers, image tags, ports, CPU, and memory limits.

Nginx

Provides one public entry point and routes dashboard, API, health, and metrics traffic.

Deployment State Machine

```
QUEUED -> BUILDING -> READY
                        \-> FAILED
READY -> STOPPED
READY -> SUPERSEDED
```

- **QUEUED:** accepted by the API.
- **BUILDING:** worker is processing the job.
- **READY:** container passed its readiness probe.
- **FAILED:** build, runtime, or health check failed.
- **STOPPED:** running deployment was stopped.
- **SUPERSEDED:** replaced by a rollback or newer version.

Security and Reliability

- JWT authentication and BCrypt password hashing.
- Stateless sessions with protected API routes.
- Ownership checks prevent cross-user project and deployment access.
- Repository input accepts HTTPS GitHub and GitLab URLs.
- Failures are persisted as status transitions and log entries.
- Redis locks make message consumption idempotent across workers.
- PostgreSQL remains the source of truth; Redis is temporary coordination storage.

Rollback

Rollback verifies project ownership, confirms that the target deployment is READY, marks the current ready deployment as SUPERSEDED, records an audit event, and restores the selected release as active.

`POST /api/v1/projects/{projectId}/deployments/{deploymentId}/rollback`

Production follow-up: route traffic by stable hostname and explicitly stop the replaced container during rollback.

Important Interview Tradeoffs

Why a modular monolith first?

It provides clear domain boundaries and simple local development without the operational cost of multiple services. The worker queue boundary allows later extraction into a separate process.

Why asynchronous deployment?

Docker builds can take minutes. Keeping them out of the HTTP request prevents timeouts, blocked server threads, and poor user experience.

Why PostgreSQL and Redis together?

PostgreSQL provides durable, transactional business data. Redis handles fast, temporary locks and state that does not need to be the system of record.

Current Demonstration

The local Compose stack contains PostgreSQL, Redis, RabbitMQ, Spring Boot, React, Nginx, Prometheus, and Grafana.

- Dashboard: <http://localhost:8081>
- Health: <http://localhost:8081/actuator/health>
- Prometheus: <http://localhost:9090>
- Grafana: <http://localhost:3000>
- RabbitMQ management: <http://localhost:15672>

Honest Project Status

Implemented and validated: authentication, project management, RabbitMQ deployment queueing, Redis coordination, PostgreSQL persistence, Docker build support, resource limits, health checks, logs, rollback state transitions, Nginx routing, metrics, Grafana, CI workflows, and Docker Compose deployment.

Next production improvements: extract the worker into a separate deployable process, implement stable hostname routing, add container cleanup policies, rate limiting, Testcontainers, GitHub App integration, TLS, VM deployment, backups, and managed secrets.

Interview Closing Statement

“I designed DeployX as a modular deployment control plane. The API handles authentication, ownership, deployment records, and job publishing, while the worker performs expensive deployment work asynchronously. PostgreSQL stores durable state, RabbitMQ decouples requests from execution, Redis prevents duplicate processing, and Docker isolates workloads. Nginx provides a single entry point, while Prometheus and Grafana provide operational visibility. I started with a modular monolith and designed clear boundaries so the worker can be extracted as the platform scales.”

Product Sales Pitch

“DeployX gives small teams a simple path from Git repository to running application without requiring them to manage deployment infrastructure manually. It combines repository-based deployments, Docker isolation, asynchronous builds, logs, health checks, rollback capability, and monitoring in one workflow. Unlike a basic deployment script, DeployX is designed around production concepts such as job queues, authorization, deployment history, failure recovery, resource limits, and observability.”

DeployX Interview Guide · Prepared for technical interviews and product discussions